

API Design

REST API Design → Representational State Transfer

An architectural style for designing networked APIs using HTTP principles. Not a protocol but a design philosophy.

REST helps :-

- * Standardize API design
- * Make APIs predictable & integration easier.
- * Improve scalability
- * Enable ~~system~~ stateless systems

REST Principles

1) Stateless → Each request must contain all required info. Server does not store session state between requests.

Why? → Stateless systems scale better. Used by Amazon, Netflix.

2) Resource based URLs :- API should represent resources (nouns) not URLs (verbs). Example :- Good :- GET /users
Bad :- GET /getUsers

3) Use Proper HTTP Methods & Status Codes.

GET → Read, POST → Create, PUT → Replace, Patch → Partial update.
200 → OK, 201 → Created, 400 → Bad request, 401 → Unauthorized
500 → Server error.

Example

Amazon's public APIs follow REST:

- Resource-based structure
- Clear versioning
- standard HTTP methods.

Best Practices in REST API Design

1) Version your API :- `/api/v1/users`

Prevents breaking clients when changes happen. Used by Stripe, Github.

2) Pagination :- Never return huge datasets.

Ex :- `GET /users ? page = 1 & limit = 20`

3) Filtering & sorting :- `GET /products ? category = electronic & sort = price`

4) Consistent naming :- Use plural nouns. `/users`, `/orders`.

5) Security :- Use HTTPS, OAuth/JWT, Rate limiting

REST in Microservices

→ Works well because :-

- Stateless
- Easy to scale
- Simple over HTTP
- Works behind load balancers.

Limitations

1) Over fetching or under-fetching data

2) Multiple round trips.

3) Less efficient for complex queries.

That's why :- Facebook introduced GraphQL & Google uses gRPC internally.

GRAPH QL

& gRPC

→

GraphQL

• In REST APIs:-

- Sometimes you get too much / too little data.
- You need multiple API calls to gather related data.

Ex:- To show user profile page:-

- 1) GET /users/123
- 2) GET /users/123/orders
- 3) GET /users/123/notifications.

Multiple round trips. Frontend becomes complicated.

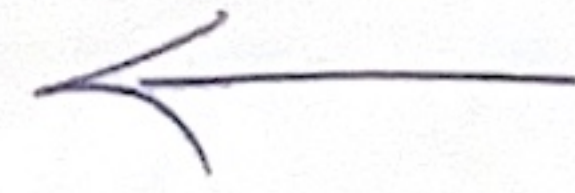
* GraphQL is a query language for APIs that allows clients to request exactly the data they need.

Working

client sends query like:-

```
query {  
  user(id: '123') {  
    name  
    email  
    orders {  
      id  
      total  
    }  
  }  
}
```

server returns exactly
that structure without
extra & missing fields.



Example

~~GitHub~~ GitHub → Uses GraphQL API publicly.

Developers can fetch repositories, issues, Pull requests.
All in one structured query.

Trade-offs

Pros

- Flexible
- Reduces API calls
- Efficient for frontend teams.
- Strong schema.

Cons

- Complex server logic
- Harder caching & rate limiting
- Requires strong monitoring
- Not always best for simple APIs.

gRPC

→ A high performance RPC (Remote procedure call) framework developed by Google.

→ It uses : HTTP/2, Protocol Buffers (Faster than REST).

→ gRPC provides :-

- High Performance
- Low Latency
- Streaming support
- Strong typing.

Ideal for :-

- Internal microservices
- Real time services.
- High-throughput systems.

Working

You define service in .proto file:

```
# service UserService {  
    rpc GetUser (UserRequest) returns (UserResponse);  
}
```

It generates client & server code automatically.

Communication happens in Binary format (Faster than JSON).

Why it is fast?

- Uses HTTP/2 (Multiplexing)
- Binary serialization (Protocol Buffers)
- Persistent connection
- Built-in streaming.

Cons of gRPC

- Harder to debug manually.
- Not browser friendly (without proxy)
- Complex tooling.
- Less human readable.

→ Better for backend to backend communication.

Asynchronous APIs

→ Client sends a request and the server does not immediately return the final result. Instead it:-

→ Acknowledges request

→ Processes work in background

→ Sends result later (or client polls)

* Imagine Uber when we request a ride.

1. User request a ride (POST/ride-request)

2. Backend searches nearby drivers.

3. Driver receives request & accepts or rejects.

(server response :- 202 Accepted
request_id : 98213)

New processing continues in background.

Client later checks :- GET/ride-status/98213

OR

Receives a websocket / push notification.

Note:- If the API waited synchronously for the entire process, the request would block & eventually timeout.

Technologies used →

• Message queues (Kafka, Rabbit MQ)

• Event streaming

• Background workers

• Websockets / call back, Polling endpoints

When Async APIs are necessary

Use when:-

• Processing takes long time

• External services involved

• Heavy computation

• Workflow spans multiple systems

• Real-time response is not required.

Example

• Document processing

• Email sending

• Payment settlement

• Video processing.

API Gateway Patterns

* Consider Netflix :- It has hundreds of microservices.

- 1) User service 2) Recommendation service 3) Playback service
4) Billing service 5) Search service.

* If the Mobile App directly called all services :-

- Too many network calls
- Security issues
- Hard version control
- Hard authentication (per microservice based)

* An API gateway is single entry point for all client requests. Clients do not talk to microservices directly.

Client → API Gateway → Microservices.

Example :- User opens Netflix homepage :-

Request :- GET /homepage

API Gateway internally calls :-

- Recommendation service
- User profile service

Then combines responses & sends one response to client.

- Content metadata service.

Responsibilities of API Gateway

API Gateway handles

- Authentication
- Request routing
- Rate limiting
- Logging
- API versioning
- Caching
- Security filtering.

* Netflix uses Zuul API Gateway.

* Amazon uses its own.

Patterns (API Gateway)

1) Backend for Frontend (BFF) :- Different clients need different APIs.

Ex:- Mobile / web API Gateway.

→ Mobile devices require lighter responses.

2) Aggregation pattern :- Gateway calls multiple services & merges response. (Explained last page :- homepage example)

3) Security Gateway :- Handles :-

- JWT validation
- OAuth authentication
- Request filtering.

4) Rate Limiting Pattern :- 100 requests / minute per user.
Used by Stripe & Twitter.

Trade Offs

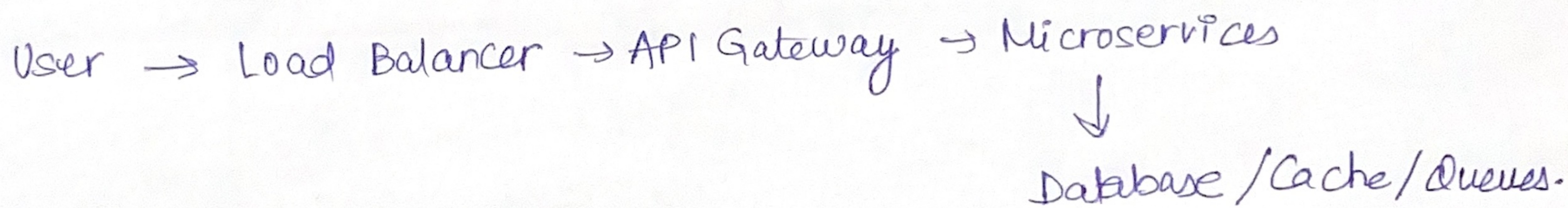
Pros:-

- Simplifies client communication
- Centralised security
- Better monitoring
- Reduces client complexity

Cons

- Adds latency
- Single point of failure if poorly designed.

Architecture



Async systems then use:

Service → Kafka → Worker → DB.

Streaming APIs for LLMs

→ When you use ChatGPT or Claude, the answer appears token by token instead of waiting 10 seconds for full response.

If system waited for full response :-

- Users would wait 5-15 seconds, UX would feel slow
- Long responses would block connections.

→ Streaming APIs allow the server to send partial responses continuously instead of sending the full response at once.

→ Request → token 1 → token 2 → token 3 →

Working

1) Most LLM platforms stream responses using :-

1) Server sent Events (SSE) • Used by OpenAI, Anthropic

Ex:- data: 'Hello'
data: "Hello, how"

2) Web Sockets :- Used for bidirectional communication • Real time chats.

3) HTTP chunked responses :- Some systems stream chunks over HTTP.

Usage

* OpenAI uses streaming so responses appear instantly while tokens are generated.

* Claude (Anthropic) uses SSE for token response.

* GitHub Copilot streams code suggestions as model generates tokens.

Trade offs

Pros

- Better user experience
- Lower perceived latency
- Works well with long responses.

Cons

- Harder error handling
- Harder caching
- Requires streaming infrastructure.